SE LAB 4

Name: Shreelakshmi G Bhat
SRN: PES2UG24CS478
Sec:H

AI Tool Used: OpenAI Codex integrated with Visual Studio Code
Model: GPT-5.6 Luna

Chat history + Code (after using the tool)

Task 1 : collision detection :

Prompt given :

I am a gamer working on a Python Pygame endless runner. The player jumps over obstacles while the obstacles move toward the player and gradually get faster.

My objective now is to fix the collision detection. At higher speeds, some obstacles can pass through the player without the collision being detected.

Please inspect my code, find the cause, and fix the collision so it stays accurate even at higher speeds. Also make sure the speed increase remains fair and playable.

Please don't rewrite the whole project. Explain what you changed and how I can test it.

Codex given answer + code:

10:51 AM
Worked for 1m 5s

The cause was that collision was checked only after each obstacle moved. At high speeds, an obstacle could cross the player between frames without overlapping its hitbox at either endpoint.

In [game_engine.py]

I added a swept collision check that accounts for both the obstacle's horizontal movement and the player's vertical movement during each frame. I also capped speed at 12 while keeping the gradual speed increase. With the existing 70-frame spawn interval, that cap preserves a useful gap between obstacles.

To try it:

1. Run `python main.py` and play through the speed increase; jump with Space, Up, or W.
2. For a quick tunneling check, temporarily set `self.speed` and `self.max_speed` to `100` in `GameEngine.__init__`. Stay on the ground and let an obstacle approach. The game should register a collision even if the obstacle crosses the player in one frame. Restore the normal values afterward.

I did not run the game.

Edited game_engine.py+58-24Review changes
UndoReview
10:52 AM

Code before using Codex :

game_engine.py

```
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)

        self.speed = 6
        # NOTE: speed keeps climbing every frame with no ceiling. See
        # Task 1 in the README for why that eventually breaks collision.
        self.speed_increase_per_frame = 0.003

        self.spawn_interval = 70  # frames between obstacle spawns
        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE,
pygame.K_UP, pygame.K_w):
            self.player.jump()

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
        if self.game_over:
            return

        self.speed += self.speed_increase_per_frame
        self.player.update()

        self._spawn_timer += 1
```

```
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y,
self.speed))

        for obstacle in self.obstacles:
            obstacle.move()
            obstacle.speed = self.speed

        # NOTE: collision is only checked against each obstacle's rect
        # *after* it has already moved for the frame. There's no check
        # for whether the obstacle's path crossed the player along the
        # way. Since self.speed has no cap, it eventually grows large
        # enough that an obstacle can cover more horizontal distance
        # in one frame than the player's own width - meaning it can be
        # entirely to the right of the player on one frame and entirely
        # to the left on the next, skipping the player's hitbox
        # completely and never registering a hit. See Task 1.
        for obstacle in self.obstacles:
            if obstacle.rect().colliderect(self.player.rect()):
                self.game_over = True
                return

        for obstacle in self.obstacles:
            if not obstacle.scored and obstacle.x + obstacle.width <
self.player.x:
                obstacle.scored = True
                self.score += 1

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]

        self.distance += self.speed

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width,
self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())
```

```
        score_text = self.font.render(f"Score: {self.score}", True, (0, 0,
0))
        screen.blit(score_text, (10, 10))

        if self.game_over and not getattr(self, "_game_over_logged",
False):
            # NOTE: no proper game-over screen yet - see Task 2 in the
README.
            print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

Code after using Codex:

```
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.player = Player(80, self.ground_y)

        self.speed = 6
        self.max_speed = 12
        self.speed_increase_per_frame = 0.003

        self.spawn_interval = 70  # frames between obstacle spawns
        self._spawn_timer = 0
```

```
        self.obstacles = []

        self.distance = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE,
pygame.K_UP, pygame.K_w):
            self.player.jump()

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
        if self.game_over:
            return

        self.speed = min(self.speed + self.speed_increase_per_frame,
self.max_speed)
        previous_player_y = self.player.y
        self.player.update()

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y,
self.speed))

        previous_obstacle_x = {}
        for obstacle in self.obstacles:
            previous_obstacle_x[obstacle] = obstacle.x
            obstacle.move()
            obstacle.speed = self.speed

        for obstacle in self.obstacles:
            if self._swept_collision(
```

```
                previous_obstacle_x[obstacle], obstacle.x,
                previous_player_y, self.player.y,
                obstacle.width, obstacle.height,
            ):
                self.game_over = True
                return

        for obstacle in self.obstacles:
            if not obstacle.scored and obstacle.x + obstacle.width <
self.player.x:
                obstacle.scored = True
                self.score += 1

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]

        self.distance += self.speed

    def _swept_collision(self, old_obstacle_x, new_obstacle_x,
                         old_player_y, new_player_y,
                         obstacle_width, obstacle_height):
        """Check overlap at any point during this frame, not just at its
end."""
        # Work in relative coordinates: the obstacle moves horizontally
and
        # the player vertically. Expand the obstacle by the player's size,
        # then test the relative motion segment against that rectangle.
        start_x = old_obstacle_x - self.player.x
        start_y = self.ground_y - obstacle_height - old_player_y
        delta_x = (new_obstacle_x - old_obstacle_x)
        delta_y = -(new_player_y - old_player_y)

        x_entry, x_exit = self._axis_overlap_times(
            start_x, delta_x, -obstacle_width, self.player.width
        )
        y_entry, y_exit = self._axis_overlap_times(
            start_y, delta_y, -obstacle_height, self.player.height
        )
        if x_entry is None or y_entry is None:
            return False
```

```
        entry_time = max(x_entry, y_entry)
        exit_time = min(x_exit, y_exit)
        return entry_time <= exit_time and exit_time >= 0 and entry_time
<= 1

    @staticmethod
    def _axis_overlap_times(position, velocity, minimum, maximum):
        """Return the time interval where a moving point lies in an
interval."""
        if velocity == 0:
            if minimum < position < maximum:
                return float("-inf"), float("inf")
            return None, None

        first = (minimum - position) / velocity
        second = (maximum - position) / velocity
        return min(first, second), max(first, second)

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width,
self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

        score_text = self.font.render(f"Score: {self.score}", True, (0, 0,
0))
        screen.blit(score_text, (10, 10))

        if self.game_over and not getattr(self, "_game_over_logged",
False):
            # NOTE: no proper game-over screen yet - see Task 2 in the
README.
            print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

Task 2 : Game Over :

Prompt 2:

I am a gamer working on a Python Pygame endless runner. I have fixed the collision issue and now want to improve what happens when the player gets hit.

My objective is to add a proper Game Over screen. When the player collides with an obstacle, the game should stop and display "GAME OVER" along with the final score inside the Pygame window.

The game should wait for user input instead of closing immediately.

Please inspect my existing code and implement this without changing the other gameplay features. Don't add replay or sound yet.

11:01 AM

Worked for 45s

Added a Game Over screen in [game_engine.py](C:/Users/Shreelakshmi Bhat/Desktop/Projects/39_endless-runner/game/game_engine.py) that dims the game and displays **GAME OVER** and the final score. The screen remains visible until a key is pressed or the window is closed. In [main.py](C:/Users/Shreelakshmi Bhat/Desktop/Projects/39_endless-runner/main.py), a key press on that screen exits the game instead of triggering a jump. No replay or sound was added.

To check it, run `python main.py`, collide with an obstacle, and confirm the screen shows the final score and stays open. Press any key or close the window to exit. I didn’t run the game.

Edited 2 files+34-12Review changes

UndoReview

game/game_engine.pygame/game_engine.py+27-8

main.pymain.py+7-4

11:02 AM

After :

```
import pygame

from .player import Player

from .obstacle import Obstacle



# Game Engine



WHITE = (255, 255, 255)

BROWN = (120, 80, 40)

DARK_GREEN = (30, 100, 30)



class GameEngine:

    def __init__(self, width, height):

        self.width = width

        self.height = height

        self.ground_y = height - 40



        self.player = Player(80, self.ground_y)



        self.speed = 6

        self.max_speed = 12

        self.speed_increase_per_frame = 0.003
```

```
        self.spawn_interval = 70  # frames between obstacle spawns
        self._spawn_timer = 0
        self.obstacles = []

        self.distance = 0
        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over_font = pygame.font.SysFont("Arial", 56, bold=True)
        self.final_score_font = pygame.font.SysFont("Arial", 32)
        self.prompt_font = pygame.font.SysFont("Arial", 22)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE,
pygame.K_UP, pygame.K_w):
            self.player.jump()

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
```

```
        if self.game_over:
            return

        self.speed = min(self.speed + self.speed_increase_per_frame,
self.max_speed)
        previous_player_y = self.player.y
        self.player.update()

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y,
self.speed))

        previous_obstacle_x = {}
        for obstacle in self.obstacles:
            previous_obstacle_x[obstacle] = obstacle.x
            obstacle.move()
            obstacle.speed = self.speed

        for obstacle in self.obstacles:
            if self._swept_collision(
                previous_obstacle_x[obstacle], obstacle.x,
                previous_player_y, self.player.y,
```

```
                obstacle.width, obstacle.height,
            ):
                self.game_over = True
                return

        for obstacle in self.obstacles:
            if not obstacle.scored and obstacle.x + obstacle.width <
self.player.x:
                obstacle.scored = True
                self.score += 1

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]

        self.distance += self.speed

    def _swept_collision(self, old_obstacle_x, new_obstacle_x,
                         old_player_y, new_player_y,
                         obstacle_width, obstacle_height):
        """Check overlap at any point during this frame, not just at its
end."""
        # Work in relative coordinates: the obstacle moves horizontally
and
        # the player vertically. Expand the obstacle by the player's size,
        # then test the relative motion segment against that rectangle.
```

```
        start_x = old_obstacle_x - self.player.x
        start_y = self.ground_y - obstacle_height - old_player_y
        delta_x = (new_obstacle_x - old_obstacle_x)
        delta_y = -(new_player_y - old_player_y)

        x_entry, x_exit = self._axis_overlap_times(
            start_x, delta_x, -obstacle_width, self.player.width
        )
        y_entry, y_exit = self._axis_overlap_times(
            start_y, delta_y, -obstacle_height, self.player.height
        )
        if x_entry is None or y_entry is None:
            return False

        entry_time = max(x_entry, y_entry)
        exit_time = min(x_exit, y_exit)
        return entry_time <= exit_time and exit_time >= 0 and entry_time
<= 1

    @staticmethod
    def _axis_overlap_times(position, velocity, minimum, maximum):
        """Return the time interval where a moving point lies in an
interval."""
        if velocity == 0:
```

```
            if minimum < position < maximum:
                return float("-inf"), float("inf")
            return None, None

        first = (minimum - position) / velocity
        second = (maximum - position) / velocity
        return min(first, second), max(first, second)

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width, self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

        score_text = self.font.render(f"Score: {self.score}", True, (0, 0, 0))
        screen.blit(score_text, (10, 10))

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 170))
```

```
            screen.blit(overlay, (0, 0))



            game_over_text = self.game_over_font.render("GAME OVER", True,
WHITE)

            final_score_text = self.final_score_font.render(

                f"Final Score: {self.score}", True, WHITE

            )

            prompt_text = self.prompt_font.render("Press any key to exit",
True, WHITE)



            screen.blit(game_over_text, game_over_text.get_rect(

                center=(self.width // 2, self.height // 2 - 55)

            ))

            screen.blit(final_score_text, final_score_text.get_rect(

                center=(self.width // 2, self.height // 2 + 5)

            ))

            screen.blit(prompt_text, prompt_text.get_rect(

                center=(self.width // 2, self.height // 2 + 55)

            ))
```

Task 3: Replay + Difficulty :

Prompt 3:

I am a gamer working on a Python Pygame endless runner. My game now has working collision detection and a Game Over screen.

My objective is to add a replay option after Game Over. I want the player to choose:

Easy
Medium
Hard
Exit

Each difficulty should have a different starting obstacle speed and spawn rate.

When replaying, the score, player position, obstacles, and game state should reset properly.

Please integrate this into my existing code without creating duplicate game loops or event handling. Don't add sound yet.

Show more
11:05 AM
Worked for 1m 4s

Added Easy, Medium, Hard, and Exit choices to the Game Over screen. Press **1**, **2**, or **3** to start a new run at that difficulty, or **4** to exit. The existing game loop and event handling remain in use.

Each replay resets the player, obstacles, score, distance, spawn timer, and game state. Starting speed and spawn interval are set per difficulty in [game_engine.py](C:/Users/Shreelakshmi Bhat/Desktop/Projects/39_endless-runner/game/game_engine.py). [main.py](C:/Users/Shreelakshmi

Bhat/Desktop/Projects/39_endless-runner/main.py) now routes events through the engine, including the Exit choice.

To check it, play until Game Over, try each difficulty, and confirm each run starts fresh. Choose **4** or close the window to exit. I didn't run the game.

Edited 2 files+54-28Review changes
UndoReview
game/game_engine.pygame/game_engine.py+51-25
main.pymain.py+3-3
11:06 AM

After(game _engine.py):

```
import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    DIFFICULTIES = {
        "Easy": (4, 100),
        "Medium": (6, 70),
        "Hard": (8, 55),
    }

    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.max_speed = 12
        self.speed_increase_per_frame = 0.003
```

```
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over_font = pygame.font.SysFont("Arial", 56, bold=True)
        self.final_score_font = pygame.font.SysFont("Arial", 32)
        self.prompt_font = pygame.font.SysFont("Arial", 22)
        self.reset("Medium")

    def reset(self, difficulty):
        """Start a fresh run using the selected starting speed and spawn
rate."""
        self.difficulty = difficulty
        self.speed, self.spawn_interval = self.DIFFICULTIES[difficulty]
        self.player = Player(80, self.ground_y)
        self._spawn_timer = 0
        self.obstacles = []
        self.distance = 0
        self.score = 0
        self.game_over = False
        self.exit_requested = False

    def handle_event(self, event):
        if event.type != pygame.KEYDOWN:
            return

        if self.game_over:
            choices = {
                pygame.K_1: "Easy",
                pygame.K_2: "Medium",
                pygame.K_3: "Hard",
            }
            if event.key in choices:
                self.reset(choices[event.key])
            elif event.key in (pygame.K_4, pygame.K_ESCAPE):
                self.exit_requested = True
            return

        if event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
            self.player.jump()

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
```

```
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
        if self.game_over:
            return

        self.speed = min(self.speed + self.speed_increase_per_frame,
self.max_speed)
        previous_player_y = self.player.y
        self.player.update()

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y,
self.speed))

        previous_obstacle_x = {}
        for obstacle in self.obstacles:
            previous_obstacle_x[obstacle] = obstacle.x
            obstacle.move()
            obstacle.speed = self.speed

        for obstacle in self.obstacles:
            if self._swept_collision(
                previous_obstacle_x[obstacle], obstacle.x,
                previous_player_y, self.player.y,
                obstacle.width, obstacle.height,
            ):
                self.game_over = True
                return

        for obstacle in self.obstacles:
            if not obstacle.scored and obstacle.x + obstacle.width <
self.player.x:
                obstacle.scored = True
                self.score += 1

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]
```

```
        self.distance += self.speed

    def _swept_collision(self, old_obstacle_x, new_obstacle_x,
                         old_player_y, new_player_y,
                         obstacle_width, obstacle_height):
        """Check overlap at any point during this frame, not just at its
end."""
        # Work in relative coordinates: the obstacle moves horizontally
and
        # the player vertically. Expand the obstacle by the player's size,
        # then test the relative motion segment against that rectangle.
        start_x = old_obstacle_x - self.player.x
        start_y = self.ground_y - obstacle_height - old_player_y
        delta_x = (new_obstacle_x - old_obstacle_x)
        delta_y = -(new_player_y - old_player_y)

        x_entry, x_exit = self._axis_overlap_times(
            start_x, delta_x, -obstacle_width, self.player.width
        )
        y_entry, y_exit = self._axis_overlap_times(
            start_y, delta_y, -obstacle_height, self.player.height
        )
        if x_entry is None or y_entry is None:
            return False

        entry_time = max(x_entry, y_entry)
        exit_time = min(x_exit, y_exit)
        return entry_time <= exit_time and exit_time >= 0 and entry_time
<= 1

    @staticmethod
    def _axis_overlap_times(position, velocity, minimum, maximum):
        """Return the time interval where a moving point lies in an
interval."""
        if velocity == 0:
            if minimum < position < maximum:
                return float("-inf"), float("inf")
            return None, None
```

```
        first = (minimum - position) / velocity
        second = (maximum - position) / velocity
        return min(first, second), max(first, second)

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width,
self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

        score_text = self.font.render(f"Score: {self.score}", True, (0, 0,
0))
        screen.blit(score_text, (10, 10))

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 170))
            screen.blit(overlay, (0, 0))

            game_over_text = self.game_over_font.render("GAME OVER", True,
WHITE)
            final_score_text = self.final_score_font.render(
                f"Final Score: {self.score}", True, WHITE
            )
            prompt_text = self.prompt_font.render(
                "Replay: 1 Easy   2 Medium   3 Hard   |   4 Exit",
                True, WHITE,
            )

            screen.blit(game_over_text, game_over_text.get_rect(
                center=(self.width // 2, self.height // 2 - 55)
            ))
            screen.blit(final_score_text, final_score_text.get_rect(
                center=(self.width // 2, self.height // 2 - 15)
            ))
            screen.blit(prompt_text, prompt_text.get_rect(
                center=(self.width // 2, self.height // 2 + 45)
```

```
            ))
```

After(main.py) :

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 800, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Endless Runner - Pygame Version")

# Colors
SKY = (200, 220, 240)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(SKY)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)
                if engine.exit_requested:
                    running = False

        engine.handle_input()
```

```
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

Task 4 : Sound:

Prompt 4 :

I am a gamer working on a Python Pygame endless runner. The game now has collision detection, Game Over, replay, and difficulty selection.

My final objective is to add sound effects for:

- Jumping
- Successfully clearing an obstacle and scoring
- Game Over

Each sound should play only once for its event.

Please use Pygame's audio functionality and tell me where to place the sound files if I need to add them. Keep the existing gameplay unchanged and make sure missing sound files don't crash the game.

Show more
11:09 AM
Worked for 1m 4s

Added optional Pygame sound effects in [game_engine.py](C:/Users/Shreelakshmi Bhat/Desktop/Projects/39_endless-runner/game/game_engine.py). Jump sound plays only when a grounded player jumps, score sound plays when an obstacle is

counted, and Game Over sound plays on the collision that ends the run. Missing files or unavailable audio leave the game running silently.

Place WAV files here:

assets/sounds/jump.wav
assets/sounds/score.wav
assets/sounds/game_over.wav

The path is based on the project location, so you can launch the game from another working directory. I didn't run the game; to check the sounds, add those files and trigger each event.

Edited game_engine.py+44-4Review changes
UndoReview
11:10 AM

After (game_engine.py):

```
from pathlib import Path

import pygame
from .player import Player
from .obstacle import Obstacle

# Game Engine

WHITE = (255, 255, 255)
BROWN = (120, 80, 40)
DARK_GREEN = (30, 100, 30)

class GameEngine:
    DIFFICULTIES = {
        "Easy": (4, 100),
        "Medium": (6, 70),
        "Hard": (8, 55),
    }

    def __init__(self, width, height):
```

```
        self.width = width
        self.height = height
        self.ground_y = height - 40

        self.max_speed = 12
        self.speed_increase_per_frame = 0.003
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over_font = pygame.font.SysFont("Arial", 56, bold=True)
        self.final_score_font = pygame.font.SysFont("Arial", 32)
        self.prompt_font = pygame.font.SysFont("Arial", 22)
        self.sounds = self._load_sounds()
        self.reset("Medium")

    def _load_sounds(self):
        """Load optional effects; the game still works if audio is
unavailable."""
        sound_dir = Path(__file__).resolve().parent.parent / "assets" /
"sounds"
        sound_files = {
            "jump": "jump.wav",
            "score": "score.wav",
            "game_over": "game_over.wav",
        }

        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init()
        except pygame.error:
            return {}

        sounds = {}
        for effect, filename in sound_files.items():
            try:
                sounds[effect] = pygame.mixer.Sound(str(sound_dir /
filename))
            except (OSError, pygame.error):
                # Missing or unsupported files simply disable that effect.
                continue
        return sounds
```

```
    def _play_sound(self, effect):
        sound = self.sounds.get(effect)
        if sound is not None:
            try:
                sound.play()
            except pygame.error:
                # Audio device errors should not interrupt gameplay.
                pass

    def reset(self, difficulty):
        """Start a fresh run using the selected starting speed and spawn
rate."""
        self.difficulty = difficulty
        self.speed, self.spawn_interval = self.DIFFICULTIES[difficulty]
        self.player = Player(80, self.ground_y)
        self._spawn_timer = 0
        self.obstacles = []
        self.distance = 0
        self.score = 0
        self.game_over = False
        self.exit_requested = False

    def handle_event(self, event):
        if event.type != pygame.KEYDOWN:
            return

        if self.game_over:
            choices = {
                pygame.K_1: "Easy",
                pygame.K_2: "Medium",
                pygame.K_3: "Hard",
            }
            if event.key in choices:
                self.reset(choices[event.key])
            elif event.key in (pygame.K_4, pygame.K_ESCAPE):
                self.exit_requested = True
            return

        if event.key in (pygame.K_SPACE, pygame.K_UP, pygame.K_w):
            if self.player.on_ground:
```

```
                self.player.jump()
                self._play_sound("jump")

    def handle_input(self):
        # Reserved for continuously-held-key input; this runner only
        # needs an edge-triggered jump, handled in handle_event.
        pass

    def update(self):
        if self.game_over:
            return

        self.speed = min(self.speed + self.speed_increase_per_frame,
self.max_speed)
        previous_player_y = self.player.y
        self.player.update()

        self._spawn_timer += 1
        if self._spawn_timer >= self.spawn_interval:
            self._spawn_timer = 0
            self.obstacles.append(Obstacle(self.width, self.ground_y,
self.speed))

        previous_obstacle_x = {}
        for obstacle in self.obstacles:
            previous_obstacle_x[obstacle] = obstacle.x
            obstacle.move()
            obstacle.speed = self.speed

        for obstacle in self.obstacles:
            if self._swept_collision(
                previous_obstacle_x[obstacle], obstacle.x,
                previous_player_y, self.player.y,
                obstacle.width, obstacle.height,
            ):
                self.game_over = True
                self._play_sound("game_over")
                return

        for obstacle in self.obstacles:
```

```
            if not obstacle.scored and obstacle.x + obstacle.width <
self.player.x:
                obstacle.scored = True
                self.score += 1
                self._play_sound("score")

        self.obstacles = [o for o in self.obstacles if not o.off_screen()]

        self.distance += self.speed

    def _swept_collision(self, old_obstacle_x, new_obstacle_x,
                         old_player_y, new_player_y,
                         obstacle_width, obstacle_height):
        """Check overlap at any point during this frame, not just at its
end."""
        # Work in relative coordinates: the obstacle moves horizontally
and
        # the player vertically. Expand the obstacle by the player's size,
        # then test the relative motion segment against that rectangle.
        start_x = old_obstacle_x - self.player.x
        start_y = self.ground_y - obstacle_height - old_player_y
        delta_x = (new_obstacle_x - old_obstacle_x)
        delta_y = -(new_player_y - old_player_y)

        x_entry, x_exit = self._axis_overlap_times(
            start_x, delta_x, -obstacle_width, self.player.width
        )
        y_entry, y_exit = self._axis_overlap_times(
            start_y, delta_y, -obstacle_height, self.player.height
        )
        if x_entry is None or y_entry is None:
            return False

        entry_time = max(x_entry, y_entry)
        exit_time = min(x_exit, y_exit)
        return entry_time <= exit_time and exit_time >= 0 and entry_time
<= 1

    @staticmethod
    def _axis_overlap_times(position, velocity, minimum, maximum):
```

```
        """Return the time interval where a moving point lies in an
interval."""
        if velocity == 0:
            if minimum < position < maximum:
                return float("-inf"), float("inf")
            return None, None

        first = (minimum - position) / velocity
        second = (maximum - position) / velocity
        return min(first, second), max(first, second)

    def render(self, screen):
        pygame.draw.line(screen, BROWN, (0, self.ground_y), (self.width,
self.ground_y), 4)

        pygame.draw.rect(screen, WHITE, self.player.rect())
        for obstacle in self.obstacles:
            pygame.draw.rect(screen, DARK_GREEN, obstacle.rect())

        score_text = self.font.render(f"Score: {self.score}", True, (0, 0,
0))
        screen.blit(score_text, (10, 10))

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 170))
            screen.blit(overlay, (0, 0))

            game_over_text = self.game_over_font.render("GAME OVER", True,
WHITE)
            final_score_text = self.final_score_font.render(
                f"Final Score: {self.score}", True, WHITE
            )
            prompt_text = self.prompt_font.render(
                "Replay: 1 Easy   2 Medium   3 Hard   |   4 Exit",
                True, WHITE,
            )

            screen.blit(game_over_text, game_over_text.get_rect(
```

```
            center=(self.width // 2, self.height // 2 - 55)
        ))
        screen.blit(final_score_text, final_score_text.get_rect(
            center=(self.width // 2, self.height // 2 - 15)
        ))
        screen.blit(prompt_text, prompt_text.get_rect(
            center=(self.width // 2, self.height // 2 + 45)
        ))
```